AFRILANG Stdlib

std/c/cryptaeslite

Bibliothèque AFRILANG `std/c/cryptaeslite` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : aeslLen, aes

`import "std/c/cryptaeslite"` · `use cryptaeslite`

- `aeslLen(s text) ? number`
- `aeslUpper(s text) ? text`
- `aeslLower(s text) ? text`
- `aeslTrim(s text) ? text`
- `aeslSplit(s text, delim text) ? list text`
- `aeslJoin(parts list text, delim text) ? text`
- `aeslReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/cryptaeslite` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : aeslLen, aes

`import "std/c/cryptaeslite"` · `use cryptaeslite`

- `aeslLen(s text) ? number`
- `aeslUpper(s text) ? text`
- `aeslLower(s text) ? text`
- `aeslTrim(s text) ? text`
- `aeslSplit(s text, delim text) ? list text`
- `aeslJoin(parts list text, delim text) ? text`
- `aeslReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptaeslite"
use cryptaeslite
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? aeslLen(s text) ? number
? aeslUpper(s text) ? text
? aeslLower(s text) ? text
? aeslTrim(s text) ? text
? aeslSplit(s text, delim text) ? list
? aeslJoin(parts list text, delim text) ? text
? aeslReplace(s text, from text, to text) ? text
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/cryptaeslite"
use cryptaeslite

create result = aeslLen(/* args */)
say result

create result = aeslUpper(/* args */)
say result

create result = aeslLower(/* args */)
say result

create result = aeslTrim(/* args */)
say result

create result = aeslSplit(/* args */)
say result

create result = aeslJoin(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/cryptaeslite"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module cryptaeslite
function aeslLen(s text) returns number
end
function aeslUpper(s text) returns text
end
function aeslLower(s text) returns text
end
function aeslTrim(s text) returns text
end
function aeslSplit(s text, delim text) returns list text
end
function aeslJoin(parts list text, delim text) returns text
end
function aeslReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/cryptaeslite` (tier complex, category complex). Exports 8 function(s): aeslLen, aeslUpper, aeslL

`import "std/c/cryptaeslite"` · `use cryptaeslite`

- `aeslLen(s text) ? number`
- `aeslUpper(s text) ? text`
- `aeslLower(s text) ? text`
- `aeslTrim(s text) ? text`
- `aeslSplit(s text, delim text) ? list text`
- `aeslJoin(parts list text, delim text) ? text`
- `aeslReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

2. Installation & import

Add the module to your program:

```
import "std/c/cryptaeslite"
use cryptaeslite
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? aeslLen(s text) ? number
? aeslUpper(s text) ? text
? aeslLower(s text) ? text
? aeslTrim(s text) ? text
? aeslSplit(s text, delim text) ? list
? aeslJoin(parts list text, delim text) ? text
? aeslReplace(s text, from text, to text) ? text
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/cryptaeslite"
use cryptaeslite

create result = aeslLen(/* args */)
say result

create result = aeslUpper(/* args */)
say result

create result = aeslLower(/* args */)
say result

create result = aeslTrim(/* args */)
say result

create result = aeslSplit(/* args */)
say result

create result = aeslJoin(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/c/cryptaeslite"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module cryptaeslite
function aeslLen(s text) returns number
end
function aeslUpper(s text) returns text
end
function aeslLower(s text) returns text
end
function aeslTrim(s text) returns text
end
function aeslSplit(s text, delim text) returns list text
end
function aeslJoin(parts list text, delim text) returns text
end
function aeslReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```